



Prova de Aptidão Profissional
Técnico de Gestão e Programação de
Sistemas Informáticos
Relatório de Projeto

Daily Budget

Elaborado por

Nome: Daniel Da Silva Ribeiro

N.º: 2223239 Triénio: 23-26

Professor orientador

Gabriel Gonçalves Benevides de Souza

26 / 06 / 2026

Índice

1 – Introdução e objetivos	3
2 – Tecnologias e recursos	4
2.1 Recursos de Hardware e Requisitos	4
2.2 Linguagens e Frameworks	5
2.3 Base de Dados e Armazenamento	5
2.4 Bibliotecas Externas e integrações	6
3 – Implementação	7
3.1 Cronograma de desenvolvimento do projeto	7
3.2 Aspetos técnicos do desenvolvimento do projeto	8
4 – Conclusão	10
5 – Bibliografia	11
6 – Anexos	12

1 – Introdução e objetivos

O Daily Budget é um programa de gestão económica feito para ajudar as pessoas a gerir melhor o dinheiro delas, fiz este programa com a intenção de haver uma aplicação grátis e pouco confusa, por vezes existe aplicações para a gestão financeira mas não as vejo para computador, então decidi fazer uma mas para computador, assim todas as pessoas têm a liberdade para usar uma app que as ajude.

Este projeto está integrado na ODS 8 (Trabalho Digno e Crescimento económico)

2 – Tecnologias e recursos

Para construir o Daily Budget, precisei de utilizar várias tecnologias e ferramentas, tanto a nível de hardware como de software. A minha estratégia foi escolher ferramentas que já conhecia das aulas para garantir um código sólido, e acrescentar tecnologias novas para tornar o projeto visualmente mais moderno e profissional.

2.1 Recursos de Hardware e Requisitos

Durante o desenvolvimento do projeto, utilizei o meu computador pessoal. Como o Visual Studio e a gestão de bases de dados exigem algum processamento na fase de criação usei uma máquina com recursos suficientes (processador multi-core e 16 GB de RAM) para não sofrer bloqueios enquanto realizava o projeto. No entanto, para o utilizador final que quiser instalar e usar a aplicação no seu dia a dia, os requisitos mínimos de hardware são bastante acessíveis. A aplicação foi feita para ser leve, pelo que basta ter:

- Um computador com Sistema Operativo Windows 10 (ou versão mais recente);
- Um processador básico (Dual-Core);
- Pelo menos 200MB de espaço livre no disco rígido. A única exigência obrigatória é ter uma ligação à internet, uma vez que funcionalidades essenciais (como o login seguro pelo Gmail e o envio de alerta) precisam de comunicar com o exterior.

2.2 Linguagens e Frameworks

- Linguagem C#(com .NET 9.0): Decidi que a linguagem principal de todo o código seria o C#. Escolhi-a porque aprendi ao longo do curso e sinto-me bastante confortável com ela para desenvolver lógicas matemáticas e cálculos de saldo. Utilizei a versão mais recente da plataforma, o .NET 9.0 porque garante que a aplicação abra de forma rápida e segura.
- Interface Gráfica com WPF (Windows Presentation Foundation): Esta foi uma justificação de design. Eu não queria usar os antigos Windows Forms porque os Ecrãs costumam ficar com um aspecto antiquado. Escolhi o WPF porque deu-me liberdade total para desenhar a aplicação com um estilo muito moderno e atrativo. O WPF (usando-a linguagem XAML) permitiu-me fazer cantos arredondados, efeitos visuais transparentes e implementar o sistema de “Data Binding”, onde as cores do programa mudam sozinhas (por exemplo, quando o orçamento fica vermelho por estar ultrapassado).

2.3 Base de Dados e Armazenamento

Para guardar todas as informações da aplicação como os dados dos utilizadores, os orçamentos e o histórico de despesas optei por utilizar o Microsoft SQL Server. Como o meu projeto lida com registos financeiros, achei bem usar um sistema de confiança para não haver o risco de perder dados ou de existirem falhas na gravação.

O SQL Server é uma base de dados relacional, o que na prática significa que a informação fica muito mais organizada. Em vez de ter tudo misturado, criei várias tabelas independentes (uma para utilizadores, outra para as despesas, outra para as categorias etc.) que se ligam entre si. Este sistema de ligações é muito útil porque evita guardar dados repetidos e garante que não há erros de coerência, como tentar associar uma despesa a um utilizador que não existe.

A ligação entre o código C# da aplicação e a base de dados foi feita através do [ADO.NET](#), utilizando o pacote Microsoft.Data.SqlClient. Escolhi esta forma de ligação porque me permitiu controlar exatamente como a aplicação comunica com a base de dados. Outro ponto muito importante foi a segurança: Utilizar este pacote e as suas ferramentas de comunicação permitiu-me proteger a aplicação contra ataques de injeções SQL. Assim garanto que um utilizador mal intencionado não consiga escrever código malicioso nas caixas de texto do programa para tentar manipular ou apagar a base de dados .

2.4 Bibliotecas Externas e integrações

Para melhorar a aplicação eu expandi o código base com várias bibliotecas externas, tais como:

-LiveCharts.Wpf (v0.9.7): O meu projeto baseia-se na entrega de estatísticas visuais. Esta biblioteca foi implementada para desenhar gráficos interativos (gráficos de tarte para as categorias e gráficos de barras para as despesas) que respondem em tempo real ao clique do rato e exibem “tooltips” detalhadas.

-MailKit (v4.17.0): Uma biblioteca “cross-platform” que substitui o antigo sistema [System.Net.Mail](#). Foi programada para interagir com servidores SMTP, permitindo à aplicação enviar automaticamente e-mails de alerta aos utilizadores sempre que o limite de uma categoria orçamental se aproxima da rutura.

-Google OAuth 2.0 (Google.Apis.Auth): A segurança moderna exige a mitigação de palavras-passe locais fracas. O sistema de login integra diretamente a API da google, permitindo aos utilizadores entrar na aplicação usando o seu próprio Gmail. Esta abordagem transfere a complexidade do “hashing” de chaves de segurança para os servidores da Google, garantindo tranquilidade máxima aos utilizadores.

Requisitos:

REQ0001: O sistema deve permitir ao utilizador registar as suas receitas : Cumprido

REQ0002: O sistema deve permitir ao utilizador registar as suas despesas: Cumprido

REQ0003: O sistema deve permitir criar e editar categorias de despesa: Cumprido

REQ0004: O sistema deve permitir associar cada despesa ou ganho a uma categoria: Cumprido

REQ0005: O sistema deve permitir visualizar o saldo atual: Cumprido

REQ0006: O sistema deve permitir consultar um resumo semanal/mensal dos gastos e poupanças : Cumprido parcialmente (foi retirada a opção de gerar o relatório mensal ou semanal e agora tem-se uma vista geral na aba relatórios)

REQ0007: O sistema deve permitir editar e eliminar registos de despesas e ganhos: Cumprido

REQ0008: O sistema deve permitir guardar os dados numa base de dados local: Cumprido

REQ0009: O sistema deve de ter a opção de fazer login: Cumprido

REQ00010: O sistema deve ter a opção de criar conta: Cumprido

3 – Implementação

3.1 Cronograma de desenvolvimento do projeto

- Fase 1 (Planeamento e protótipos) O projeto iniciou-se com a definição dos objetivos principais, levantamento dos requisitos e análise funcional. Durante esta fase, foram elaborados os primeiros protótipos da interface gráfica, que serviam de base para todo o desenvolvimento posterior.
- Fase 2 (Design e Front-End) Esta fase consistiu no desenvolvimento da interface gráfica do utilizador (UI). Ao invés da tecnologia inicialmente prevista no pré-projeto (Windows Forms), optou-se pela transição para WPF, garantindo uma experiência muito mais moderna e fluida. Foram criadas as janelas principais, painéis de registo e os menus de navegação.
- Fase 3 (Base de Dados e Back-End) Procedeu-se à criação e integração da base de dados no Microsoft SQL Server para armazenar a informação importante, como receitas, despesas e categorias. Foi também implementado o código responsável por fazer a ligação lógica entre a interface visual e a base de dados.
- Fase 4 (Implementação da lógica) Neste período programou-se o “cérebro” do projeto. Foram desenvolvidas as funcionalidades cruciais e a matemática do sistema, nomeadamente os cálculos do saldo final, a geração de relatórios de histórico e a gestão de compras através da “WishList”
- Fase 5 (Testes e validações) Este período foi dedicado ao controlo da qualidade do projeto fazendo testes no sistema, verificou-se também o funcionamento de todas as funções do programa para garantir que tudo funciona como devido.
- Fase 6 (Documentação e apresentação) Revi o projeto pela última vez antes de entregar tudo e trabalhei agora no relatório final .

3.2 Aspetos técnicos do desenvolvimento do projeto

A nível da estrutura do código, esforcei-me por manter o projeto o mais organizado possível, separando bem as responsabilidades. Dividi a solução do Visual Studio em várias pastas fundamentais: as “Vistas” (Onde guardo as janelas desenhadas em XAML), os “Modelos” (que representam a lógica dos dados) e a pasta “Dados” (que gera a comunicação direta com o SQL Server). Isto ajudou-me imenso a não me perder no meio de tanto código e facilitou muito a manutenção do programa ao longo dos meses.

O “corpo” do projeto funciona com base em três classes principais que desenvolvi:

- A Classe “Transacao” é aqui que ficam registados todos os movimentos de dinheiro. Criei propriedades para guardar o “Valor”, a “Data” e a “Categoria”. Para facilitar a apresentação visual da aplicação, adicionei a lógica que muda automaticamente o ícone (para receitas ou despesas) no momento em que é carregado para a lista.
- Classe “orcamentoCategoria” Esta é a classe responsável por controlar se o utilizador está a gastar demasiado num determinado orçamento. Ela compara o limite que a pessoa definiu com o que já foi gasto nesse mês. Se o limite for ultrapassado, criei um sistema de “Data binding” que faz a interface gráfica reagir no mesmo instante, pintando a barra de progresso de vermelho e mostrando um aviso visual.
- Classe “ItemDesejado”(WishList) A funcionalidade de planear compras foi um dos pontos altos da interação da aplicação. Esta classe avalia constantemente o saldo atual da conta e o preço do item que o utilizador quer comprar. Através de um cálculo matemático automático, converte essa diferença percentual no tamanho dinâmico da barra de progresso.
- Desafios técnicos e resolução de problemas: Durante o desenvolvimento final, deparei-me com desafios de código difíceis. Um dos maiores foi o facto da aplicação congelar/crashar sempre que tentava carregar os relatórios a partir da base de dados. Para resolver isto, tive de implementar programação assíncrona no C# usando (“async” e “await”). Isto permitiu-me pôr a aplicação a procurar os dados num processo secundário e “invisível” mantendo a navegação nos menus sempre rápida e sem interrupções.

- Outro erro crucial que tive de resolver recentemente ocorreu no sperador de Categorias. A aplicação entrava num erro fatal (crash) e ia abaixo quando a percentagem das barras de orçamento recebia valores que ultrapassavam os limites da interface XAML (por exemplo, quando se gastava muito mais do que o estipulado). Para corrigir isto, tive de rever o “Data binding” e implementar limites de segurança matemáticos no código, garantindo que a interface gráfica nunca cede a valores extremos, tornando a versão final da aplicação estável.

4 – Conclusão

Chegar ao fim deste projeto e ver o programa totalmente funcional é algo que considero bom e gratificante. Foi um percurso exigente, com desafios e horas a rever código, mas no final sinto que valeu a pena. O meu grande objetivo com a PAP era criar uma aplicação que fosse não apenas fiável a gerir contas e orçamentos mas sim que realmente ajudasse as pessoas tal como eu já tinha referido. Sinto que esse objetivo tenha sido cumprido com sucesso: Entreguei uma ferramenta que realmente ajuda qualquer utilizador a organizar a sua carteira e a planear as suas compras de forma inteligente.

Claro que o desenvolvimento não foi isento de dificuldades. Como referi nos aspetos técnicos, tive de enfrentar problemas que me fizeram um pouco de confusão, como o erro que fazia a aplicação ir abaixo nas categorias quando a barra de limite matemática falhava. Resolver estas situações exigiu de mim uma certa quantidade de pesquisa e paciência pois também tive de usar conceitos que não aprendi nas aulas. No entanto, tentar perceber o porquê de um erro acontecer e conseguir corrigi-lo com sucesso foi o que mais me fez evoluir. Terminei este projeto com felicidade e orgulho no resultado final e com a certeza que adquiri mais competências técnicas para o futuro.

Trabalho que posso fazer no futuro:

- “Integração Mobile Cloud”: Expansão do projeto para uma aplicação móvel. Dado que a arquitetura em Web API já existe e está a funcionar no computador, os utilizadores poderiam instalar uma versão no telemóvel para registar um café ou almoço de imediato enquanto estão no exterior, sincronizando a despesa na base de dados central.
- “Contas conjuntas e orçamentos familiares”: Implementar a lógica permissões e multiutilizador. Isto permitiria que duas contas (por exemplo marido e esposa) partilhassem acesso visual à mesma “conta da casa”, analisando orçamentos comuns em tempo real, independente do dispositivo de onde entraram.
- “Módulo de análise Fiscal”: Possibilidade da aplicação emitir, com um clique, ficheiros PDF estruturados e faturados, facilitando a entrega anual do IRS ou o envio de dados de escritórios de contabilidade.

5 – Bibliografia

[Windows Presentation Foundation for .NET documentation | Microsoft Learn](#)

<https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api>

6 – Anexos



